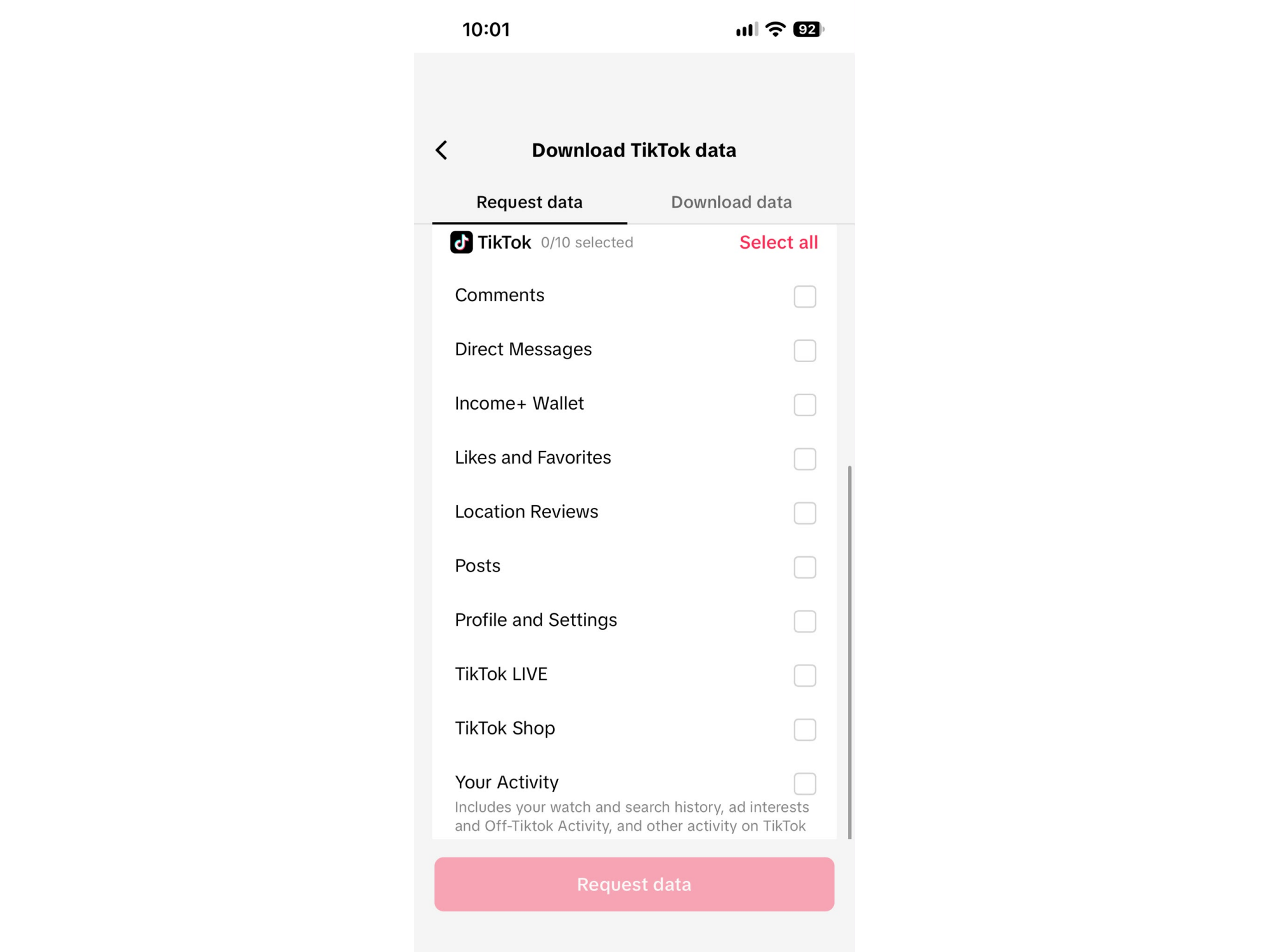
Large Language Models for Social Science Research

Session 2. Structured Data with an Agent

Hugo Subtil

PhD Workshop, University of Zurich

October 28, 2026


10:01
92
Download TikTok data
Request data
Download data
TikTok 0/10 selected
Select all
Comments
Direct Messages
Income+ Wallet
Likes and Favorites
Location Reviews
Posts
Profile and Settings
TikTok LIVE
TikTok Shop
Your Activity
Includes your watch and search history, ad interests and Off-Tiktok Activity, and other activity on TikTok
Request data

Today's Session Overview (9:00–12:00)

9:00–9:45 How to Work with an Agent (45 min)

- Last Week, and What to Keep in Mind
- How I Actually Work
- Tokens and Models
- Where I Also Use AI

9:45–10:25 Block 1. Document and Specify (40 min), then a 15-min break

- Data README, codebook, analysis plan (06)

10:40–11:15 Block 2. Clean, Merge, Describe (35 min)

- Scripts that check themselves; statistics, plots, first insights (07)

11:15–11:45 Block 3. Audit and Decide (30 min)

- A fresh reviewer (08), then one step by hand (09)

11:45–12:00 Debrief and Next Session (15 min)

Last Week

What we saw, and what to keep in mind

Last Week, in One Slide

The argument. An AI system now writes economics papers on its own (APE). Generating a paper is almost free; verifying one is not. The bottleneck moved from *doing* to *judging*, and the chain has to end with a human who understands.

The position. Delegate the work. Keep the skill to check it, and the responsibility for it. Three rules:

1. You delegate execution, never understanding.
2. Read before you accept.
3. Stop when you no longer know what it is doing.

The tool. A folder that remembers: `README.md` (what the project is), `CLAUDE.md` (how the agent behaves), `PROJECT_HISTORY.md` (where we stand). Raw data read-only, prompts versioned, every session ends with a commit.

What to Keep in Mind

This is still an open debate. How much to delegate, and how much to check, is not settled. Reasonable people draw the line in different places.

Since last week, I asked many colleagues one question:

Where is your limit on review?

What do you read line by line, and what do you let go?

Today, one answer I find convincing, from Germain Gauthier (Bocconi). Not *the* answer: a workable one, which you can adapt.

How I Actually Work

Inspired by Germain Gauthier (Bocconi)

How I Use AI

- AI is part of pretty much everything I do on my laptop.

Some use cases

- A quick overview of papers I am unfamiliar with
- Writing code, organising project folders, updating results in a paper
- Proof-checking that the code, the paper and the slides are consistent
- Discussing ideas and analyses while I work on a project
- Improving the clarity of my writing and presentations

Pen and paper when I need to fully understand something (theory, statistics), or for mental clarity (a to-do list, planning ahead).

How I Do Not Use AI

The one thing not to do:

"Write me a full paper and a full slide set on this vague idea while I go swim in the Limmat."

- **First, current AIs are terrible at this when left unattended.** It is a very iterative process, and you need to be in the loop.
- **Second, you will present the paper and the results.** Being in the loop is how you understand what you are doing.

The journey is almost as important as the destination in research.

The Setup: One Project Folder

Germain's agent is connected to five tools

Dropbox	project files and data
GitHub	code history and repository
Overleaf	paper and slides
HPC cluster	heavy computation
Gmail	co-authors' comments

The project folder

`my-project/`	
`overleaf/`	local copy of the paper
`code/`	code repository
`data/`	raw and processed data
`notes/`	working notes

How it runs

- The agent runs from the project folder.
- He talks to it in the browser, to see PDFs and figures as they are produced.
- The local copy of the paper and the Overleaf project are always in sync.

Principle 1: You Are in Charge

- Agents overproduce, expand the scope, and overreact to details.
- Do not let them run unattended for long (overnight), unless the task is very well defined: for example, monitoring jobs on a cluster and relaunching the ones that fail.
- Stay hands-on: in the loop, correcting course, making the judgement calls.
- Being hands-on is also how you keep understanding what you are doing, and keep a map of it in your head, even when you no longer write the code.

Principle 2: Accuracy, Not Speed or Cost-Cutting

- Run the models at maximum reasoning. For *everything*.
- A mistake in a paper costs far more than extra tokens or waiting time.
- The difference in intelligence between reasoning levels is large.
- More reasoning does not remove the need for verification.

A Typical Loop

1. **Specify the task in writing.** Sample, equation, estimator, clustering, expected output: as in a detailed pre-analysis plan. Germain writes it in Overleaf; you, in `notes/analysis_plan.md`.

2. **A first agent implements it.** It flags ambiguities *before* coding: there is high value in being on the same page.

3. **Other agents review it in a fresh context.** They read the plan, the code and the outputs, and list the discrepancies.

4. **You decide what to revise or accept.** A reviewer asked to find problems always finds something. Resolve, rerun, repeat until satisfied.

Not exotic: Anthropic's own guide to Claude Code recommends the same steps.

The Typical Failure: Silent Decisions

A typical failure mode is the AI making consequential decisions without asking. For example, dropping some observations.

On data, it looks like this

- rows lost in a merge (inner join instead of left join)
- a missing code (−9, 99) read as a value
- a duplicate removed
- an identifier read as a number (`01001` becomes `1001`)
- missing values treated without asking: blank incomes set to 0, or every respondent who refused dropped. The regression runs on another sample, and the coefficient moves.

Two defences

- Every script prints the number of rows before and after each step. A change the plan does not explain stops the agent.
- I always ask the reviewing agents to check for this too.

Two Phases: Prototyping, Then Sharpening

Prototyping phase:

- It is an exploratory phase.
- I do not review the code or the prose.
- I treat these outputs as probable but provisional.

Sharpening phase:

- If I think there is something there, I get involved in the writing and coding process.
- I audit the code, paying special attention to the number of rows in the dataset and results that "look weird". I have many independent agents check the code.
- I rewrite the paper and work with AI to iterate on the visuals.
- I take responsibility for the results and claims.

Communication with Co-authors

Keep three distinctions explicit.

Origin	what the AI produced; what I produced
Status	what is exploratory; what is polished
Responsibility	what I have not checked; what I am ready to stand behind

Your co-authors will not look at exploratory and polished results in the same way, and will not make the same comments. AI use and understanding vary across co-authors: be transparent, and take responsibility for your work.

Tokens and Models

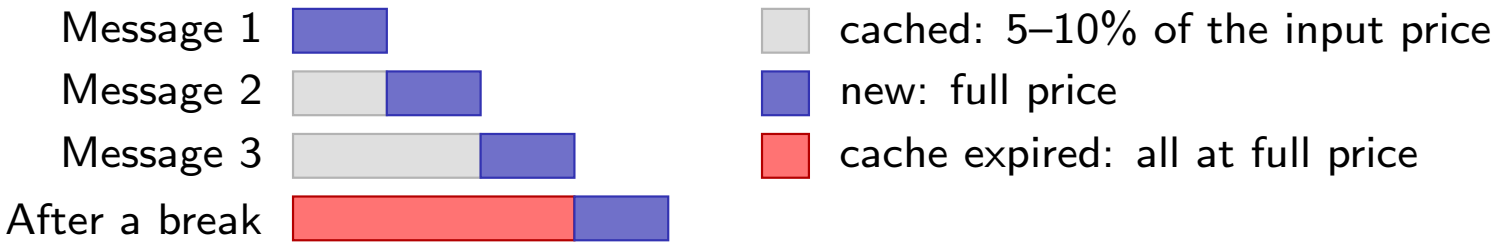
What Principle 2 costs, and where you can save

What Costs Tokens

- **The model has no memory between messages.** Each message re-sends everything: the instructions, `CLAUDE.md`, every file read, every output, every previous exchange. Your 40th message carries the 39 before it.
- **The cache makes this re-reading cheaper, not free** (next slide).
- **Effort multiplies tokens**: more files read, more checks, more thinking.
- **A bigger model costs more per turn.**
- **Subagents** spend tokens while you are not typing.

Subscription: tokens are quota. API key: tokens are money.
`/usage` shows where yours went.

The Cache: Why Pauses and Switches Cost



- **The cache keeps what was already processed**, for a while. The timer restarts at every message: one hour on a subscription, five minutes with an API key or once you pay for extra usage.
- **A pause longer than that empties it.** The next message re-processes the whole conversation at full price. Back from lunch: a fresh session is cheaper than a long old one.
- **Changing the start of the conversation empties it too.** Each model has its own cache: `/model` in the middle of a session re-reads everything. Choose the model at the start.

`/usage` shows your cache hit rate. Sources: Anthropic, prompt caching (API and Claude Code documentation), October 2026.

Model and Effort: Two Dials

Model: how capable. A bigger model understands more: unfamiliar libraries, ambiguous instructions, subtle bugs. It costs more per token.

Effort: how thorough. The same model works longer: it reads more files, thinks more, checks its own work before answering. It costs more tokens.

When the answer is wrong, ask which one it is.

- It *did not know*: it misread an ambiguous variable, or did not know how a package behaves. Change the **model**.
- It *did not try*: it skipped a file, did not rerun the code, did not check. Raise the **effort**.

`/model` to see and change the model. `/effort` for the effort: `low`, `medium`, `high`, `xhigh`, `max`.

Which Model, Which Effort, for Which Task

Task	**Model**	**Effort**	**Why**
Rename, reformat a table, commit message	Sonnet	`low`	you check each result anyway
Prototyping: exploratory scripts, quick plots	Sonnet	`medium`	provisional by design
Writing: the paper, memos, the analysis plan	Fable	`high`	long, ambiguous tasks
Code that enters the paper: cleaning, merging, estimation	Opus	`high`	errors hide in cases that are easy to overlook
Independent review	Opus	`xhigh`	the last check before yours

Default in Claude Code: Opus 5.5 at `medium`. Fable is the most expensive per token: select it with `/model fable`, and keep it for what needs it.

Save on Context, Not on Reasoning

- **One task, one session.** `/clear` between unrelated tasks costs nothing and loses nothing: the project's memory is in the folder (`PROJECT_HISTORY.md`), not in the chat.
- **Watch the first minute.** `Esc` as soon as it drifts; `/rewind` to go back. After two failed corrections: `/clear` and one better prompt, not a third correction.
- **Never paste data.** The agent writes a script that prints summaries (types, counts, missing values); only the summaries enter the context. It also protects confidential data.
- **Point, do not paste.** `@code/03_merge.py`, not the file's content. PDFs: only the pages you need.
- **A short** `CLAUDE.md`: under about 200 lines. Long procedures go in `prompts/`.
- **A long task:** `/compact` with a focus ("keep the merge decisions"). Back after more than an hour: a fresh session costs less.

Save on context. Never on reasoning for what enters the paper.

Where I Also Use AI

Beyond the syllabus: small things that save hours

A Literature Watch, on a Schedule

Every Monday at 8:00, a scheduled task opens a fresh session in my project folder. It looks for what came out last week on my topics, and writes a one-page digest in `notes/literature_watch/`.

The instruction

Search for working papers and articles published in the last seven days on [my topics]. Read the abstracts only. Write `notes/literature_watch/<date>.md`: at most ten papers, each with authors, title, link, one sentence on the finding, one on the link with `README.md`. Mark the three closest to this project. Never add a file to `literature/`. Never list a paper you could not open.

How

- Claude Desktop, **Code** tab, **Scheduled**, **New task**: the instruction, the folder, **Weekly**. Or ask in any session: "every Monday at 8am, ...".
- It runs only while the app is open and the computer awake; a missed run is caught up once. A cloud task runs with the computer off, but without your local files.
- I still click every link. A paper that does not exist is the classic error.

Other Easy Wins for Researchers

- **Before you submit**: rerun the replication package from a clean copy, and check every number in the paper and the slides against the outputs.
- **References**: check that every citation in a draft exists and matches its DOI; clean the `.bib`.
- **Referee reports**: one table with every comment, where the revision answers it (page, line), and a draft response that you rewrite.
- **Calls and applications**: check a proposal against the call (page limits, required sections, eligibility) before you send it.
- **Data releases**: a scheduled task that tells you when a new survey wave or a new election file is published.
- **Teaching**: exercises and solutions from your own slides; a check that slides, worksheet and code say the same thing.

Same rule as everything else today: the agent drafts, you check.

Your Project, Three Blocks

9:45 – 11:45

I show, you do, we check

The Example Project: One Step Further

In Session 1, at the end of the demo, the agent downloaded five presidential elections (2002–2022) and cleaned them into one table of municipalities. Its checks passed.

But it took six decisions alone: the scope, the overseas codes, Paris, the far-right candidate, Zemmour, blank ballots. And municipalities merged between elections: the same code does not always mean the same territory.

Today's question

Did the far-right vote grow more between 2002 and 2022
where the population declined?

Three sources, three geographies

- The five elections: each in the municipal geography of its year
- INSEE historical series, censuses 1968–2015: the geography of 2017
- INSEE history of municipalities: every merger, split and change of code

What You Leave With at Noon

In your folder:

- `data/raw/README.md` and `notes/codebook.md`: where each file comes from, what each variable means
- `notes/analysis_plan.md`, in your words, with its open questions settled
- numbered scripts in `code/` that print their row counts and check themselves, and `code/run_all.py`
- a cleaned dataset in `data/processed/`, summary statistics, two figures, and `notes/insights.md`
- a review written by a fresh session, and one step you audited by hand
- an entry in `PROJECT_HISTORY.md` and a commit

And one thing you will have seen: a decision the agent took on your data without asking. Finding it is Block 3. It is the only step you cannot skip.

Block 1. Document and Specify (9:45–10:25)

I show (10 min). The example project. Prompt 06: a script inspects the three sources without changing them. The agent completes the data README, writes the codebook, interviews me, and writes the plan.

You do (25 min)

1. Your data in `data/raw/`. No data? The example pack, in the course repository (`sessions/02/`).
2. Copy prompts 05 to 09 and `notes/templates/` from the course repository (`template/`) into your folder. A new session. Prompt 06.
3. Answer its questions. What you do not know yet goes to "Open questions". It asks; it does not decide.
4. Read the codebook and the plan before you say "ok".

Survey data: the codebook also records the weights, the filter questions, and the codes for refusal, don't know and not asked.

We check (5 min). Two codebooks on screen. A variable whose missing-value codes nobody documented? A merge with no rule for the rows that do not match?

Scripts, Not Chats

- The agent writes code into the project. It does not answer in the chat: a number in a chat is a claim nobody can rerun.
- Every table and figure has a script behind it. `python3 code/run_all.py` rebuilds everything from `data/raw/`.
- Coding assistance, three uses:
  - Writing: one script per step, each saying what it reads, writes and drops.
  - Debugging: paste the error, and ask for the cause before the fix.
  - Documenting: a docstring at the top of each script, the codebook for the variables.
- Reading the code stays your job. Ask the agent to explain it.

Developers learning a new library with an AI assistant scored 50% on a quiz, against 67% for those coding by hand. Those who asked the AI for explanations scored about as well as by hand (Shen and Tamkin 2026).

Checks to Request

Silent errors: bad merges, dropped observations, miscoded missing values, mislabelled variables. Three checks catch most of them. Ask for them in the code, not in the chat.

Row counts, before and after every step

```
merge: 1,000 rows -> 980; unmatched: 20 left, 5 right
```

Assertions, which stop the script when they fail

```
assert df["code_commune"].is_unique
assert df["code_commune"].str.len().eq(5).all()
assert df["share_fnrn"].between(0, 1).all()
assert merged["votes"].sum() == raw["votes"].sum()
```

Cross-tabulations, after every recode and every merge

```
pd.crosstab(df["q12_raw"], df["q12_clean"], dropna=False)
pd.crosstab(merged["_merge"], merged["region"])
```

Block 2. Clean, Merge, Describe (10:40–11:15)

I show (10 min). `/clear`, then prompt 07. The agent lists what the plan leaves open; I answer. It writes the scripts, with their checks, then the summary statistics, two figures and its first insights. We read the row counts first, the figures last.

You do (20 min)

1. `/clear`: the plan is in the folder, not in the chat. Prompt 07.
2. Answer its list of ambiguities before it writes any code.
3. Read in this order: the row counts, the assertions, the cross-tabulations, then the figures and the insights.
4. Rows that change without the plan saying so: paste the step in the shared document.

We check (5 min). The shared document: who lost rows, at which step, and was it in the plan?

When to Trust, When to Intervene

Trust a step when a check exists for it, it passed, and you can rerun it.

Intervene when

- a number moved and nobody can say why
- a match is right on the code and wrong on the territory, the wave or the period
- a label says one thing and the code computes another
- the result looks too clean

Audit one step by hand (prompt 09)

1. Choose the riskiest step: a merge, a recode, a filter.
2. Read its code. Ask the agent to explain each line.
3. Follow three units from the raw file to the final table: one ordinary, one edge case, one dropped or unmatched.
4. Read their values in the raw file yourself, and recompute one number by hand.

Block 3. Audit and Decide (11:15–11:45)

I show (10 min). I close the session and open a fresh one. Prompt 08: a reviewer that did not see the code being written. Then prompt 09: I audit one step by hand, three municipalities that became one.

You do (15 min)

1. Close the session. Open a new one. Prompt 08. It runs for several minutes: do not wait.
2. Prompt 09 on that step, in a second session. Read three units in the raw data yourself. Not optional.
3. For each point of the review and of your audit: fix it, or accept it and write down why. Rerun `code/run_all.py`.
4. Prompt 05: the entry in `PROJECT_HISTORY.md`, and the commit.

We check. That is the debrief.

Debrief

Three screens. A codebook. A row-count trail. A step audited by hand, with what it found.

The decisions the agent took alone today. Read from the shared document. The list is the lesson.

One question. The table you would have sent your supervisor an hour ago: which status, exploratory or polished, would it have carried?

Before you leave. Ask the agent to copy the section "Row counts and silent decisions" of `template/CLAUDE.md` into yours. Then add one line of your own that would have caught today's error.

Next Session: 4 November 2026

Session 3: Building Datasets from APIs, and Classic Text-as-Data

- Collecting structured and unstructured data through APIs, with the agent writing the collector
- Information extraction without an LLM: dictionaries, embeddings
- Bring one source you would like to collect from: an API, a website, a set of documents
- Keep working in your folder. End every session with prompt 05.

Carry-over question

An API returns 10,000 documents and the agent reports that the collection is complete. How would you know?

References

Gauthier, G. (2026). Using AI in Research Workflows. Slides, Bocconi University, September 2026. pinchofdata.github.io/germaingauthier. The section "How I Actually Work" follows his slides closely.

Anthropic (2026). Best Practices for Claude Code. code.claude.com/docs/en/best-practices.

Anthropic (2026). Model Configuration: Aliases and Effort Levels. code.claude.com/docs/en/model-config.

Anthropic Support (2026). Models and Usage Limits in Claude Code. support.claude.com.

Hallie, L. (2026). Choosing a Claude Model and Effort Level in Claude Code. *Claude Blog*, 7 July 2026.

Shen, J. H. and Tamkin, A. (2026). How AI Assistance Impacts the Formation of Coding Skills. Anthropic Research, 29 January 2026. anthropic.com/research.